

PAYMENT TRACK, DAY 2: the course owner's share, FRONTEND

Angular 16 + Material, Midnight theme. **2 new components, 1 new service, 1 new model file, 1 new module, 3 small edits.** Do the backend guide of Day 2 first.

0. What the user sees

- **Instructor:** a new link **Earnings** in the menu. The page shows three numbers (**Total earned, Waiting for payout, Paid to you**), a line "The platform keeps 20% of each sale", a table of every sale (what the student paid, the platform share, your share, status Waiting or Paid) and a table of payouts received with the bank or UPI reference.
- **Admin:** a new link **Payouts**. The page lists each instructor who is owed money (name, e-mail, mobile, number of sales, amount), a box for the bank or UPI reference and the button **Mark as paid** (disabled until the reference has 3 characters). A pop-up asks "Mark as paid?" with the name, the amount and the reference. Afterwards the row disappears and the payout appears under **Payout history**.
- Customers see nothing new today (invoices come on Day 3). A customer or an instructor who types `/admin-payouts` is sent away by the existing `AuthGuard`.

Honest status. Real browser, pretend backend: the earnings and payouts screens passed in the same **39-check** run that also covers Day 3 (instructor sees 399.20 waiting after a 499 sale at 20%, admin marks it paid, instructor then sees 0.00 waiting and 399.20 paid, customers and instructors cannot open Payouts, **0 page errors**). I also built the app in exactly the Day 2 state (before the invoice screens exist) and it compiles. The real backend was not connected.

1. Edits and new files

1. `constant.ts` : add two lines inside the `API` object

Next to the other payment lines:

```
INSTRUCTOR_EARNINGS: '/api/instructor/earnings',  
ADMIN_PAYOUTS: '/api/admin/payouts',
```

`angularapp/src/app/models/billing.model.ts` (ts)

```new file (Day 3 replaces it with a longer version) */\* Money is always a whole number of paise (100 paise = 1 rupee). Divide by 100 to show rupees. I*

```
export interface EarningRow { id: number; courseId: number; courseTitle: string; paymentId: number; grossPaise: number; feePaise: number; ownerPaise: number; status: 'PENDING' | 'PAID_OUT'; createdAt: string; }
```

```
export interface PayoutView { id: number; instructorUserId: number; instructorName: string; amountPaise: number; earningsCount: number; reference: string; paidByEmail: string; paidAt: string; }
```

```
export interface EarningsSummary { totalPaise: number; pendingPaise: number; paidOutPaise: number; commissionPercent: number; earnings: EarningRow[]; payouts: PayoutView[]; }
```

```
export interface PayableRow { instructorUserId: number; name: string; email: string; mobile: string; pendingPaise: number; pendingCount: number; }
```

```
```angularapp/src/app/services/earnings.service.ts``` (ts)
```

```

```new file
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { API, API_URL } from '../constant';
import { EarningsSummary, PayableRow, PayoutView } from '../models/billing.model';

/** Instructor earnings, and the admin's payouts. */
@Injectable({ providedIn: 'root' })
export class EarningsService {
 private apiUrl = API_URL;

 constructor(private http: HttpClient) {}

 mine(): Observable<EarningsSummary> {
 return this.http.get<EarningsSummary>(`${this.apiUrl}${API.INSTRUCTOR_EARNINGS}`);
 }

 payable(): Observable<PayableRow[]> {
 return this.http.get<PayableRow[]>(`${this.apiUrl}${API.ADMIN_PAYOUTS}/payable`);
 }

 history(): Observable<PayoutView[]> {
 return this.http.get<PayoutView[]>(`${this.apiUrl}${API.ADMIN_PAYOUTS}`);
 }

 pay(instructorUserId: number, reference: string): Observable<PayoutView> {
 return this.http.post<PayoutView>(`${this.apiUrl}${API.ADMIN_PAYOUTS}`, { instructorUserId, reference });
 }
}

```

#### angularapp/src/app/components/instructor-earnings/instructor-earnings.component.ts (ts)

```

```new file import { Component, OnInit } from '@angular/core'; import { EarningsSummary } from
'../models/billing.model'; import { EarningsService } from '../services/earnings.service';

```

```

/* An instructor's sales: how much was earned, what waits for payout, what was already paid. */ @Component({ selector:
'app-instructor-earnings', templateUrl: './instructor-earnings.component.html', styleUrls: ['./instructor-
earnings.component.css'] }) export class InstructorEarningsComponent implements OnInit { summary: EarningsSummary
| null = null; loaded = false; loadError = '';

```

```

constructor(private api: EarningsService) {}

```

```

ngOnInit(): void { this.api.mine().subscribe({ next: s => { this.summary = s; this.loaded = true; }, error: () => { this.loaded =
true; this.loadError = 'Could not load your earnings. Please try again.'; } }); }

```

```

```angularapp/src/app/components/instructor-earnings/instructor-earnings.component.html``` (html)

```new file
<div class="page">
  <div class="page-head">
    <div>
      <h2>My earnings</h2>
      <p>What students paid for your courses, minus the platform share. The admin pays you out and records it here.</p>
    </div>
  </div>

  <div class="form-error" role="alert" id="earningsError" *ngIf="loadError"><mat-icon>error_outline</mat-icon>{{ loadError
}}</div>

  <ng-container *ngIf="summary">
    <div class="stats">
      <div class="stat"><span>Total earned</span><strong id="totalEarned">{{ summary.totalPaise / 100 |
currency:'INR':'symbol':'1.2-2' }}</strong></div>
      <div class="stat wait"><span>Waiting for payout</span><strong id="pendingAmount">{{ summary.pendingPaise / 100 |
currency:'INR':'symbol':'1.2-2' }}</strong></div>
      <div class="stat ok"><span>Paid to you</span><strong id="paidAmount">{{ summary.paidOutPaise / 100 |
currency:'INR':'symbol':'1.2-2' }}</strong></div>
    </div>
    <p class="note">The platform keeps {{ summary.commissionPercent }}% of each sale. You get the rest.</p>

    <h3>Sales</h3>
  </ng-container>
</div>

```

```

<p class="empty" id="noSales" *ngIf="!summary.earnings.length">No sales yet. When a student pays for one of your
courses, it appears here.</p>
<div class="table-card" *ngIf="summary.earnings.length">
  <table class="data">
    <thead><tr><th>Date</th><th>Course</th><th class="num">Student paid</th><th class="num">Platform share</th><th
class="num">Your share</th><th>Status</th></tr></thead>
    <tbody>
      <tr *ngFor="let e of summary.earnings" [attr.id]='earning' + e.id">
        <td>{{ e.createdAt | date: 'd MMM y' }}</td>
        <td>{{ e.courseTitle }}</td>
        <td class="num">{{ e.grossPaise / 100 | currency:'INR':'symbol':'1.2-2' }}</td>
        <td class="num">{{ e.feePaise / 100 | currency:'INR':'symbol':'1.2-2' }}</td>
        <td class="num"><strong>{{ e.ownerPaise / 100 | currency:'INR':'symbol':'1.2-2' }}</strong></td>
        <td><span class="chip" [ngClass]="e.status === 'PAID_OUT' ? 'ok' : 'wait'">{{ e.status === 'PAID_OUT' ? 'Paid'
: 'Waiting' }}</span></td>
      </tr>
    </tbody>
  </table>
</div>

<h3>Payouts received</h3>
<p class="empty" id="noPayouts" *ngIf="!summary.payouts.length">You have not received a payout yet.</p>
<div class="table-card" *ngIf="summary.payouts.length">
  <table class="data">
    <thead><tr><th>Date</th><th class="num">Amount</th><th>Sales covered</th><th>Reference</th></tr></thead>
    <tbody>
      <tr *ngFor="let p of summary.payouts" [attr.id]='payout' + p.id">
        <td>{{ p.paidAt | date: 'd MMM y' }}</td>
        <td class="num"><strong>{{ p.amountPaise / 100 | currency:'INR':'symbol':'1.2-2' }}</strong></td>
        <td>{{ p.earningsCount }}</td>
        <td>{{ p.reference }}</td>
      </tr>
    </tbody>
  </table>
</div>
</ng-container>
</div>

```

angularapp/src/app/components/instructor-earnings/instructor-earnings.component.css (CSS)

```

```new file / Instructor: earnings (Midnight) / .stats { display: grid; grid-template-columns: repeat(auto-fit, minmax(210px,
1fr)); gap: 16px; margin-bottom: 8px; } .stat { display: flex; flex-direction: column; gap: 4px; padding: 18px 20px; border-
radius: var(--radius); background: var(--surface); border: 1px solid var(--line); box-shadow: var(--shadow); } .stat span {
color: var(--muted); font-size: 13.5px; } .stat strong { font: 800 26px var(--display); color: var(--ink); } .stat.wait strong {
color: var(--gold-text); } .stat.ok strong { color: var(--ok); } .note { margin: 6px 0 24px; color: var(--muted); font-size: 14px;
} h3 { margin: 26px 0 10px; font-size: 18px; } .empty { padding: 18px 0; color: var(--muted); } .num { text-align: right; }
.chip { padding: 3px 12px; border-radius: 999px; font: 700 12.5px Inter, sans-serif; } .chip.wait { background: var(--gold-
soft); color: var(--gold-text); } .chip.ok { background: rgba(52, 211, 153, .14); color: var(--ok); }

```

```

```angularapp/src/app/components/admin-payouts/admin-payouts.component.ts``` (ts)

```new file
import { Component, OnInit } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { forkJoin } from 'rxjs';
import { PayableRow, PayoutView } from '../../../models/billing.model';
import { EarningsService } from '../../../services/earnings.service';
import { ConfirmDialogComponent } from '../../../confirm-dialog/confirm-dialog.component';

/**
 * Admin: who is owed money. The admin pays the instructor outside the app (bank or UPI), types the transaction
 * reference here and presses Mark as paid. That moves every pending sale of that instructor to Paid.
 */
@Component({
 selector: 'app-admin-payouts',
 templateUrl: './admin-payouts.component.html',
 styleUrls: ['./admin-payouts.component.css']
})
export class AdminPayoutsComponent implements OnInit {
 payable: PayableRow[] = [];
 history: PayoutView[] = [];
 refs: { [instructorUserId: number]: string } = {};

```

```

loaded = false;
loadError = '';
actionError = '';
actionOk = '';
busyId: number | null = null;

constructor(private api: EarningsService, private dialog: MatDialog) {}

ngOnInit(): void { this.load(); }

load(): void {
 forkJoin({ payable: this.api.payable(), history: this.api.history() }).subscribe({
 next: r => { this.payable = r.payable; this.history = r.history; this.loaded = true; },
 error: () => { this.loaded = true; this.loadError = 'Could not load the payouts. Please try again.'; }
 });
}

refOk(r: PayableRow): boolean {
 const v = (this.refs[r.instructorUserId] || '').trim();
 return v.length >= 3 && v.length <= 100;
}

markPaid(r: PayableRow): void {
 if (!this.refOk(r) || this.busyId !== null) { return; }
 this.dialog.open(ConfirmDialogComponent, {
 data: {
 title: 'Mark as paid?',
 message: 'You are recording that ' + r.name + ' was paid ' + (r.pendingPaise / 100).toFixed(2) + ' rupees.
Reference: ' + this.refs[r.instructorUserId].trim() + '.',
 confirmText: 'Mark as paid', cancelText: 'Cancel', icon: 'payments'
 }
 }).afterClosed().subscribe(yes => { if (yes) { this.pay(r); } });
}

private pay(r: PayableRow): void {
 this.busyId = r.instructorUserId;
 this.actionError = '';
 this.actionOk = '';
 this.api.pay(r.instructorUserId, this.refs[r.instructorUserId].trim()).subscribe({
 next: p => {
 this.busyId = null;
 this.actionOk = 'Recorded: ' + r.name + ' was paid ' + (p.amountPaise / 100).toFixed(2) + ' rupees.';
 delete this.refs[r.instructorUserId];
 this.load();
 },
 error: err => {
 this.busyId = null;
 this.actionError = err.error?.message || 'Could not record the payout. Please try again.';
 }
 });
}

trackByUser(_: number, r: PayableRow): number { return r.instructorUserId; }
}

```

**angularapp/src/app/components/admin-payouts/admin-payouts.component.html** (html)

```new file

Instructor payouts

All student money arrives in the platform's Razorpay account. Pay the instructor by bank or UPI, then record it here.

```

error_outline{{ loadError }}
error_outline{{ actionError }}

```

```

{{ actionOk }}

```

To pay

Nobody is owed anything right now.

| Instructor | Contact | Sales | Owed | Bank / UPI reference | |
|--------------|---------------------------------|----------------------|--|---|--|
| {{ r.name }} | {{ r.email }}
{{ r.mobile }} | {{ r.pendingCount }} | {{ r.pendingPaise / 100 currency:'INR':'symbol':'1.2-2' }} | <input type="text" value="Transaction id"/> | {{ busyId === r.instructorUserId ? 'Saving...' : 'Mark as paid' }} |

Payout history

No payouts recorded yet.

| Date | Instructor | Amount | Sales | Reference | Recorded by |
|--|------------------------|---|-----------------------|-------------------|---------------------|
| {{ p.paidAt date: 'd MMM y, h:mm a' }} | {{ p.instructorName }} | {{ p.amountPaise / 100 currency:'INR':'symbol':'1.2-2' }} | {{ p.earningsCount }} | {{ p.reference }} | {{ p.paidByEmail }} |

```

**`angularapp/src/app/components/admin-payouts/admin-payouts.component.css`** (css)

```new file
/* Admin: payouts (Midnight) */
h3 { margin: 24px 0 10px; font-size: 18px; }
.empty { padding: 14px 0; color: var(--muted); }
.muted { color: var(--muted); font-size: 13.5px; }
.num { text-align: right; }
.ref { width: 100%; min-width: 150px; padding: 9px 12px; border-radius: 10px; border: 1px solid var(--line); background: var(--bg); color: var(--ink); font: 400 14.5px Inter, sans-serif; }
.ref:focus { outline: none; border-color: var(--gold); }

```

### angularapp/src/app/modules/billing/billing.module.ts (ts)

```

```new file (Day 3 replaces it) import { NgModule } from '@angular/core'; import { RouterModule, Routes } from '@angular/router'; import { SharedModule } from '../shared/shared.module'; import { AuthGuard } from '../components/authguard/authguard.guard'; import { ROLES } from '../constant'; import { INSTRUCTOR_ROLE } from '../models/instructor.model'; import { InstructorEarningsComponent } from '../components/instructor-earnings/instructor-earnings.component'; import { AdminPayoutsComponent } from '../components/admin-payouts/admin-payouts.component';

```

```

const routes: Routes = [ { path: 'instructor/earnings', component: InstructorEarningsComponent, canActivate: [AuthGuard], data: { roles: [INSTRUCTOR_ROLE] } }, { path: 'admin-payouts', component: AdminPayoutsComponent, canActivate: [AuthGuard], data: { roles: [ROLES.ADMIN] } } ];

```

/ * Earnings for instructors and payouts for admins. (Day 3 adds the invoice screens to this same module.) *
Import it in AppModule BEFORE AppRoutingModuleModule so these routes are registered before the "" fallback. */
 @NgModule({ declarations: [InstructorEarningsComponent, AdminPayoutsComponent], imports: [SharedModule, RouterModule.forChild(routes)] }) export class BillingModule {} ````

2. app.module.ts : two small edits

1. Add this import with the other module imports: `import { BillingModule } from './modules/billing/billing.module';`
2. In the `imports: [...]` list add `BillingModule` **before** `AppRoutingModule` (after `VideoModule`):

```

VideoModule,
BillingModule,
AppRoutingModule

```

The order matters: the router uses the first route that matches, and `AppRoutingModule` ends with a "match everything" route.

3. components/navbar/navbar.component.html : two links, each in two places

The file has a desktop menu and a phone menu (`mat-menu-item`). In the **ADMIN** block, after the `Videos` link add:

```
<a routerLink="/admin-payouts" routerLinkActive="active">Payouts</a>
```

and in the phone menu, after the admin `Videos` link:

```
<a mat-menu-item routerLink="/admin-payouts">Payouts</a>
```

In the **INSTRUCTOR** block, after its `Videos` link add:

```
<a routerLink="/instructor/earnings" routerLinkActive="active">Earnings</a>
```

and in the phone menu, after the instructor `Videos` link:

```
<a mat-menu-item routerLink="/instructor/earnings">Earnings</a>
```

2. Run it and test it

1. `npm start`. It must compile with no red error.
2. Make a sale (Day 2 backend, section 4b), then log in as that **instructor**: **Earnings** shows the sale with **Waiting** status, and the three numbers add up.
3. Log in as **admin**: **Payouts** lists the instructor and the amount owed. Type a reference such as `UPI-9981`, press **Mark as paid**, confirm. The success line appears and the row moves to **Payout history**.
4. Log in as the instructor again: **Waiting** is 0, **Paid** shows the amount, and the payout with its reference is listed.

Acceptance checklist

- [] Instructor menu has **Earnings**; admin menu has **Payouts**
- [] An instructor course sale shows the owner's share (price less commission) as **Waiting**
- [] An admin-made course sale shows nothing for any instructor
- [] **Mark as paid** stays disabled until the reference has 3 or more characters
- [] After a payout the instructor sees **Paid** and the reference; the admin row is gone
- [] A customer opening `/admin-payouts` is redirected

3. If something goes wrong (frontend)

What you see	Why	Fix
<code>API.INSTRUCTOR_EARNINGS</code> does not exist	edit 1 missed	add the two lines to <code>constant.ts</code>
Click on Earnings shows the home page	<code>BillingModule</code> is after <code>AppRoutingModule</code>	move it before
Earnings page: "Could not load your earnings"	backend not updated, or you are not logged in as INSTRUCTOR (403)	check the backend log and the role
Payouts page empty although a sale was paid	the course is an admin course (no owner), or the earning failed to write	check <code>course_owners</code> and the backend log
"Nothing is pending for this instructor" when marking paid	already paid out (maybe in another tab)	reload the page